COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/20/2026	**DATE SUBMITTED:**	10/22/2026
NAME:	Al Twain D. Pasion	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 14 Algorithm Complexity

1. PROCEDURE OUTPUT

What procedures did you perform?

CODE

.h file

```
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    T temp[right - left + 1];

    int i = left;
    int j = middle + 1;
    int temp_pos = 0;

    while (i <= middle && j <= right) {
        if (arr[i] < arr[j]) {
            temp[temp_pos] = arr[i];
            i++;
            temp_pos++;
        } else {
            temp[temp_pos] = arr[j];
            j++;
            temp_pos++;
        }
    }
    while (i <= middle) {
        temp[temp_pos] = arr[i];
        i++;
        temp_pos++;
    }
    while (j <= right) {
        temp[temp_pos] = arr[j];
        j++;
        temp_pos++;
    }
    int k = 0;
    for (int i = left; i <= right; i++) {
        arr[i] = temp[k];
        k++;
    }
}
```

```cpp
template <typename T>
void mergeSort(T arr[], int left, int right) {

    /*
    If N=1, Arr is already sorted
    If N>1,
    Left = 0, right = N-1
    Find middle = (left + right)/2
    Call merge_sort(Arr,left,middle) =>sort first half recursively
    Call merge_sort(Arr,middle+1,right) => sort second half recursively
    Call merge(Arr, left, middle, right) to merge sorted arrays in above steps.
    Exit
    */

    if (left >= right) {
        return;
    }
    int middle = (left + right) / 2;

    mergeSort(arr, left, middle);
    mergeSort(arr, middle + 1, right);
    merge(arr, left, middle, right);
}
```

```cpp
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot_value = arr[high];
    int boundary = low - 1;

    for (int i = low; i <= high-1; i++) {
        if (arr[i] < pivot_value) {
            boundary++;
            std::swap(arr[i], arr[boundary]);
        }
    }
    std::swap(arr[high], arr[boundary+1]);
    return boundary+1;
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    /*
        if(low < high)
        pivot = partition (A,low,high);
        quicksort(A,low,pivot-1)
        quicksort(A,pivot+1,high)
    */

    if (low < high) {
        int pivot = partition(arr, low, high);
        quickSort(arr, low, pivot-1);
        quickSort(arr, pivot + 1, high);
    }
}
```

main file

```cpp
#include <iostream>
#include <algorithm>
#include <random>
#include <iterator>
#include <chrono>
#include "sorting2.h"

void print(int arr[], int size) {
    for (int i = 0; i < size; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << "\n";
}

int main() {
    const int size = 20;
    int arr1[size];
    int arr2[size];

    const int seed = 67;
    std::mt19937 gen(seed);
    std::uniform_int_distribution<int> give( a: 1, b: 101);

    for (int i = 0; i < size; i++) {
        arr1[i] = give( [&] gen);
    }

    std::copy( first: std::begin( [&] arr1), last: std::end( [&] arr1), result: arr2);

    std::cout << "\n---- UNSORTED ARRAYS (same seed generation) ----\n";
    print(arr1, size);
    print(arr2, size);

    std::cout << "\n---- QUICK SORT TIMING ----\n";
    auto t1 :time_point<system_clock> = std::chrono::high_resolution_clock::now();
    quickSort(arr1, low: 0, high: size-1);
    auto t2 :time_point<system_clock> = std::chrono::high_resolution_clock::now();

    auto duration :long long = std::chrono::duration_cast<std::chrono::nanoseconds>( d: t2 - t1 ).count();
    std::cout << "Time it took: " << duration << " nanoseconds\n";

    std::cout << "\n---- MERGE SORT TIMING ----\n";
    auto t3 :time_point<system_clock> = std::chrono::high_resolution_clock::now();
    mergeSort(arr2, left: 0, right: size-1);
    auto t4 :time_point<system_clock> = std::chrono::high_resolution_clock::now();

    auto duration2 :long long = std::chrono::duration_cast<std::chrono::nanoseconds>( d: t4 - t3 ).count();
    std::cout << "Time it took: " << duration2 << " nanoseconds\n";

    std::cout << "\n---- SORTED ARRAY ----\n";
    print(arr1, size);

    int worst1[size];
    int worst2[size];
```

```
    int worst1[size];
    int worst2[size];

    for (int i = 0; i < size; i++) {
        worst1[i] = i;
        worst2[i] = i;
    }

    std::cout << "\n---- WORST CASE: ALREADY SORTED ARRAY (size " << size << ") ----\n";

    std::cout << "\n---- QUICK SORT TIMING (WORST CASE) ----\n";
    auto w1 :time_point<system_clock> = std::chrono::high_resolution_clock::now();
    quickSort(worst1, low: 0, high: size - 1);
    auto w2 :time_point<system_clock> = std::chrono::high_resolution_clock::now();

    auto worstDuration1 :long long = std::chrono::duration_cast<std::chrono::nanoseconds>( d: w2 - w1 ).count();
    std::cout << "Time it took: " << worstDuration1 << " nanoseconds\n";

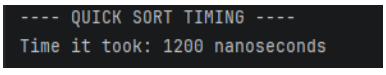
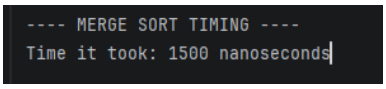
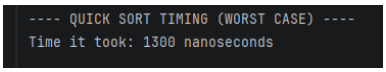
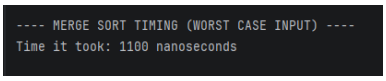
    std::cout << "\n---- MERGE SORT TIMING (WORST CASE INPUT) ----\n";
    auto w3 :time_point<system_clock> = std::chrono::high_resolution_clock::now();
    mergeSort(worst2, left: 0, right: size - 1);
    auto w4 :time_point<system_clock> = std::chrono::high_resolution_clock::now();

    auto worstDuration2 :long long = std::chrono::duration_cast<std::chrono::nanoseconds>( d: w4 - w3 ).count();
    std::cout << "Time it took: " << worstDuration2 << " nanoseconds\n";
}
```

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	BEST CASE	WORST CASE	ANALYSIS
QUICK SORT	O(n log n)	O(n^2)	The best case for quick sort is O (n log n) since it splits the array near the middle each time about "log n" levels with "n" per level. The worst case happens when the pivot is either the low extreme or the high extreme. This does not cut the array in half in the most efficient way.
MERGE SORT	O(n log n)	O(n log n)	The best and worst case for merge sort is always O (n log n) since it always splits the array in half no matter what by "log n" levels and merging it per level in "n" times. The difference here is the extra memory for making a sub-array.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

	QUICK SORT	MERGE SORT
BEST CASE		
WORST CASE		

ANALYSIS
We can see clearly that for the best case, they performed similarly since their theoretical best case is both O(n log n). For the worst case, this is where we can see differences in time complexity. Quick sort now has O(n^2) while Merge sort is still O (n log n) which is now faster.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be theoretical and empirical analysis.

For the theoretical analysis, the deliverable must be:

- Pseudocode Analysis
- Time and Space Complexities

PSEUDOCODE

Binary Search Pseudocode

```
Algorithm BinarySearch(array, size, target)
   low = 0
   high = size - 1

   While low <= high
      middle = (low + high) / 2

      If array[middle] == target Then
         Print "Search Success"
         Return
      Else If array[middle] > target Then
         high = middle - 1
      Else
         low = middle + 1
      End If
   End While

   Print "Search Failed"
End Algorithm
```

Linear Search Pseudocode

```
Algorithm LinearSearch(array, size, target)
   i = 0
   While i < size
      If array[i] == target Then
         Print "Search Success"
         Return
      End If
      i = i + 1
   End While

   Print "Search Failed"
End Algorithm
```

THEORETICAL ANALYSIS (PSEUDOCODE & TIME AND SPACE COMPLEXITY ANALYSIS)
In concept, the difference between binary search and linear search is the set up and searching technique used. Binary searching requires the starting list to be already sorted while linear search does not need it to be. In the pseudocode, we can see that the binary search uses the “divide-and-conquer” technique in searching whereas it divides the list in half every iteration until it narrows it down to one element. Linear searching uses the “sequential” technique where it checks every element one by one from start to finish. For the time complexity, binary searching has the best and worst case of O(1) and O(log n) respectively, while linear search has the best and worst case of O(1) and O(n). The space complexity is technically the same, using the same amount of memory for either algorithm because they don’t make sub-arrays and such.

For the empirical analysis, the deliverable must be the filled-out table and your analysis.

INPUT SIZE 1,000

```
Not Found!
1200 nanoseconds for Linear Searching for input size: 1000
Not Found!
200 nanoseconds for Binary Searching for input size: 1000
```

INPUT SIZE 10,000

```
Not Found!
6400 nanoseconds for Linear Searching for input size: 10000
Not Found!
400 nanoseconds for Binary Searching for input size: 10000
```

INPUT SIZE 100,000

```
Not Found!
59600 nanoseconds for Linear Searching for input size: 100000
Not Found!
500 nanoseconds for Binary Searching for input size: 100000
```

ANALYSIS

For the searching algorithms used: Binary and Linear, their worst case is simply if the element they're searching for is not in the array. When I ran the command we can see that the results do match the theoretical perspective. For linear searching, as the input size grows larger and larger, the longer the function takes to process. Same goes with binary searching, since its worst-case time complexity is faster than linear's, its processing is much quicker but it still takes longer the more the input size increases. While binary searching is much more efficient in finding an element compared to linear, the list still needs to be sorted. Linear searching works on any type of data list, unsorted or not, but it takes more time as input size grows.

CONCLUSION

In this activity, we had in-depth learning of algorithm complexity; time and space. For the procedure, we analyzed sorting techniques, specifically quick and merge sort. We observed that the quick sort and merge sort are at the same processing speed when both are in best case. And in the worst case, merge sort is faster than quick sort in terms of theoretical and experimental. For the supplementary, I tested searching algorithms specifically; binary and linear. I tested the algorithms by increasing the input size of the list. From what I observed, binary is much more efficient than linear but it needs a sorted list while linear is good for any data list. Their speeds differ but their processing times increase while input size also increases. Linear search increases more exponentially while binary search is nearly 100 nanoseconds per decimal place. I need to improve on my outside knowledge for this activity, I need to search for ways to get random values in a range and in a fixed seed. Overall, this activity broadens my knowledge on time and space complexity of algorithms. Theoretically, it's perfect but in experimental it's almost approximate.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.